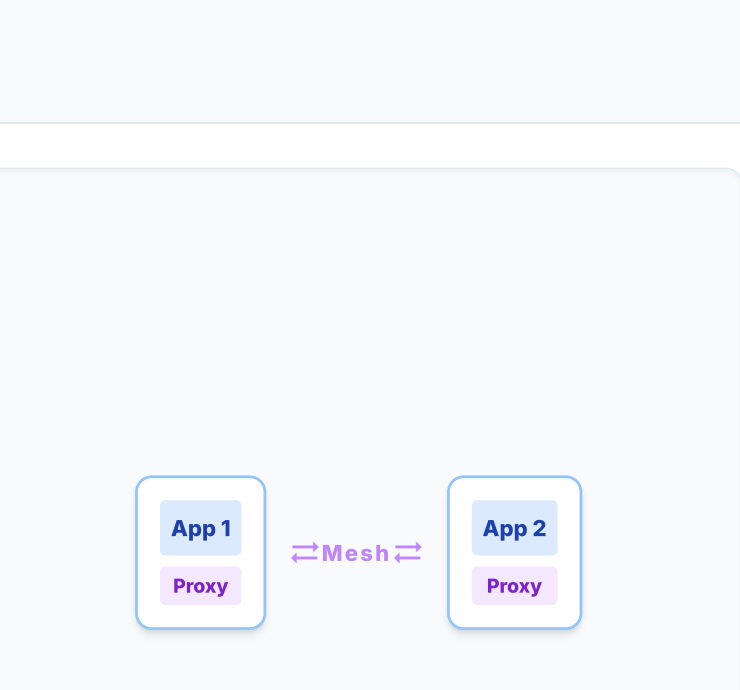
Maîtriser Istio

Le Service Mesh de Référence : Sécuriser, Connecter et Observer

02. Qu'est-ce qu'un Service Mesh ?

- **Couche Transparente :** S'ajoute au-dessus de vos applications distribuées (Kubernetes).
- **Zéro Modification :** La logique réseau n'est plus codée dans les microservices.
- **3 Piliers :** Sécurité (mTLS), Gestion du Trafic, et Observabilité.



03. Pourquoi adopter Istio ?

Standardisation

Unifie la gestion réseau avec une seule API, quel que soit le langage de programmation de vos microservices.

Performance (Envoy)

Propulsé par le proxy Envoy (C++), garantissant une latence minimale et une robustesse éprouvée.

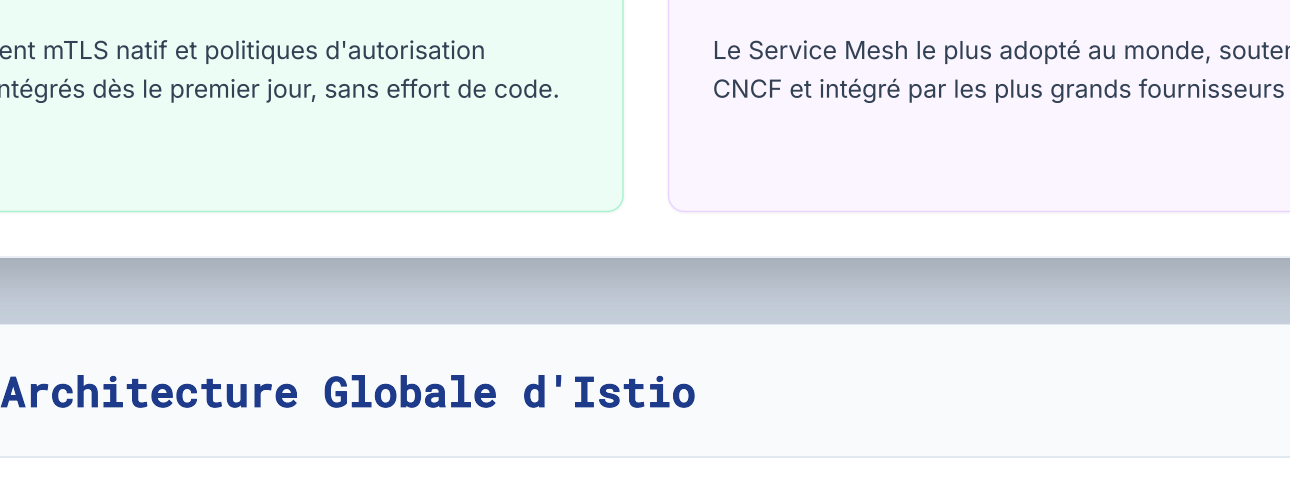
Sécurité Zero-Trust

Chiffrement mTLS natif et politiques d'autorisation strictes intégrés dès le premier jour, sans effort de code.

Écosystème & Communauté

Le Service Mesh le plus adopté au monde, soutenu par la CNCF et intégré par les plus grands fournisseurs Cloud.

04. L'Architecture Globale d'Istio



Control Plane (Istiod)
Traduit les configurations (YAML) et programme les proxies.

Data Plane (Envoy)
Intercepte 100% du trafic entre les microservices.

05. Flexibilité du Data Plane

Istio propose aujourd'hui deux approches :

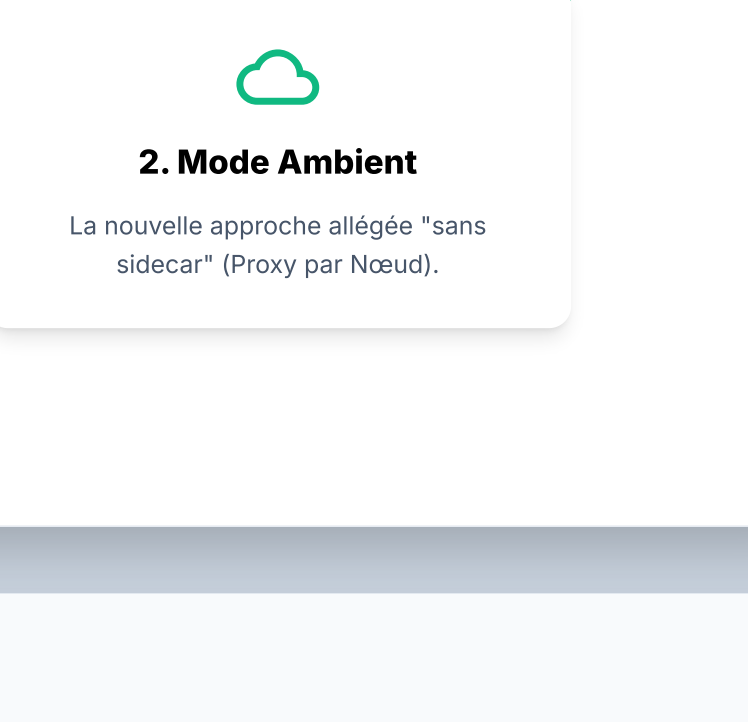
1. Mode Sidecar
L'approche historique, robuste et testée en production (Proxy par Pod).

2. Mode Ambient
La nouvelle approche allégée "sans sidecar" (Proxy par Nœud).

06. Le Mode Sidecar (Classique)

> Un proxy Envoy est **"injecté"** automatiquement dans chaque Pod applicatif.

- Traite le trafic Réseau (L4) et Applicatif (L7).
- **Avantage :** Sécurité maximale (chaque workload a ses clés isolées).
- **Inconvénient :** Coût en ressources (CPU/RAM) élevé si on a des centaines de Pods.



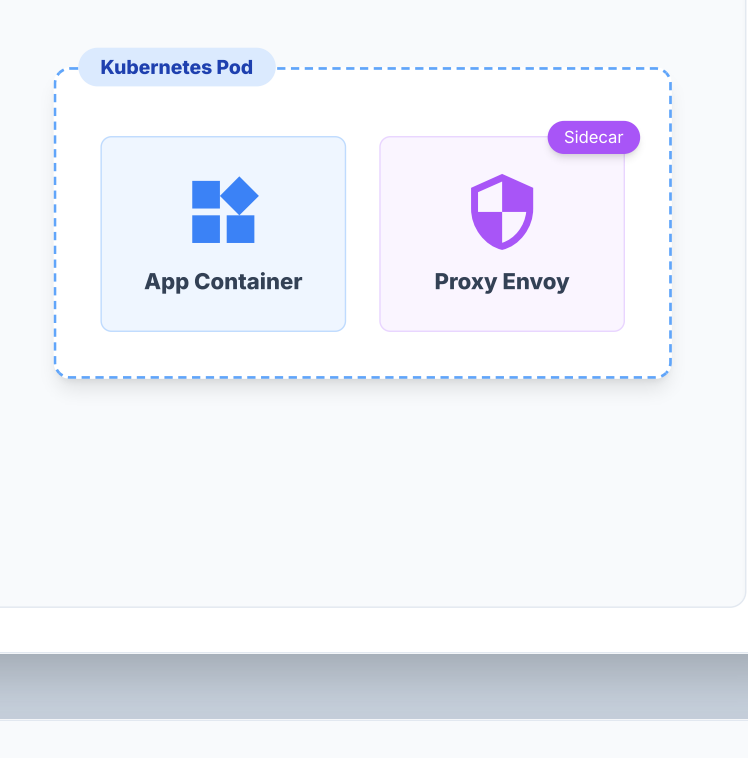
07. Le Mode Ambient (Moderne)

Léger & Transparent

Proxy L4 mutualisé par nœud (**ztunnel**) + Proxy L7 par namespace (**waypoint**).

✓ **Aucun redémarrage de Pod requis**

Réduit drastiquement l'empreinte mémoire sur le cluster.



08. Sidecar vs Ambient : Lequel choisir ?

Critère	Mode Sidecar	Mode Ambient
Ressources	Gourmand (un proxy par pod)	**Très léger (mutualisé)**
Latence (p90)	0.6ms - 0.8ms	**L4 : 0.1ms - 0.2ms**
Déploiement	Redémarrage des pods requis	**Transparent**
Support	**Stable, Multi-cluster, VMs**	Stable, Cluster unique

09. Couche 4 (L4) vs Couche 7 (L7)

Couche 4 (Réseau)

L4

Traitement : TCP, IP, Ports.

Coût CPU : Très faible.

Sécurité : Chiffrement mTLS entre les pods.

Observabilité : Octets transférés, volume de paquets.

Couche 7 (Applicatif)

L7

Traitement : HTTP, gRPC, Headers, URIs.

Coût CPU : Plus élevé (Parsing profond).

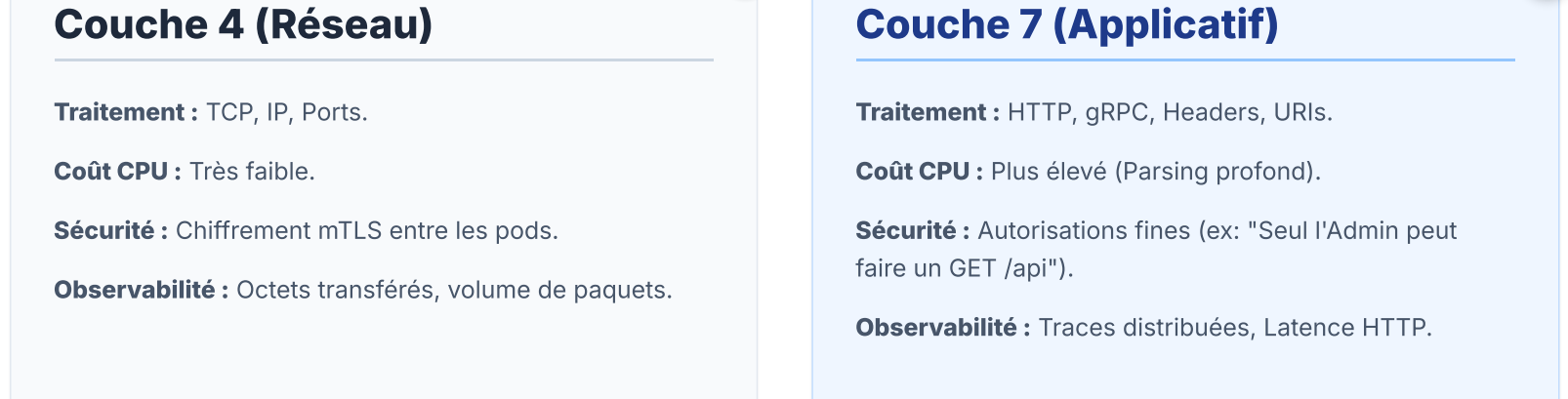
Sécurité : Autorisations fines (ex: "Seul l'Admin peut faire un GET /api").

Observabilité : Traces distribuées, Latence HTTP.

10. Gérer le Trafic : La Feuille de Route

Comment Istio route une requête ?

Découpler l'infrastructure du réseau permet une gestion des releases ultra-sécurisée.



1. **Gateway**
   La porte d'entrée.
   (L4/L6 : Ports, TLS)
2. **Virtual Service**
   Le Routeur Intelligent.
   (Où aller ? Ex: v1 ou v2)
3. **Destination Rule**
   La Politique d'arrivée.
   (Load Balancing, Subsets)
4. **Workloads**
   Vos Pods Kubernetes
   (Exécutent le code)

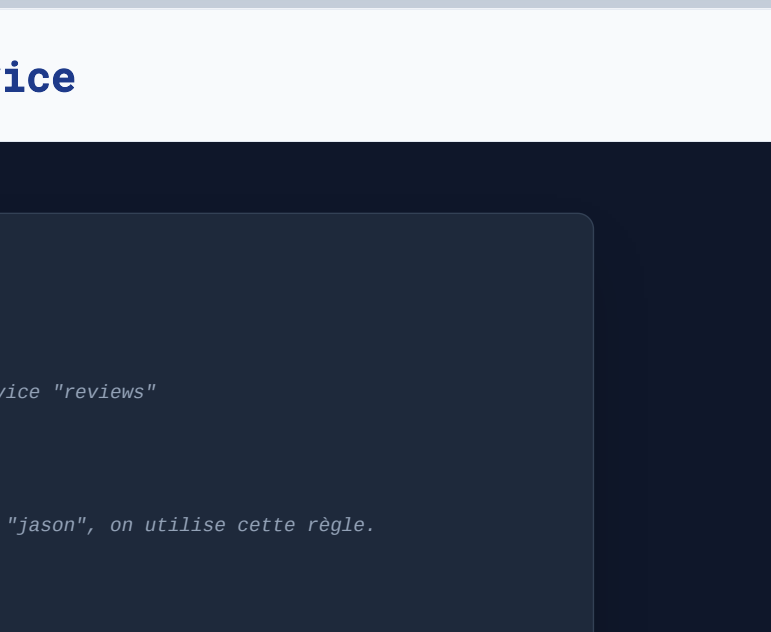
11. Virtual Services : Le Routage Intelligent

Le "GPS" de votre architecture

Le Virtual Service déconnecte fortement le nom de l'URL demandé par le client des pods réels qui vont traiter la requête.

Les Fonctionnalités clés :

- **Le bloc "hosts" :** Définit l'adresse ciblée par le client.
- **Le bloc "match" :** Filtre les requêtes (par URI, headers spécifiques, ou ports).
- **Le bloc "route" :** Indique la destination réelle et le "Subset" (ex: la version V2).
- **Priorité :** Les règles sont lues de haut en bas. La première correspondante est appliquée.



12. Code en action : Virtual Service

```yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
spec:
  # 1. HOSTS : S'applique aux requêtes visant le service "reviews"
  hosts:
  - reviews

  http:
  # 2. MATCH : Si le header HTTP contient exactement "jason", on utilise cette règle.
  - match:
    - headers:
        end-user:
          exact: jason
    route:
    - destination:
        host: reviews
        subset: v2

  # 3. ROUTE PAR DÉFAUT : Règle lue en dernier (sans condition 'match').
  # Tout le reste du trafic ira vers le subset v3.
  - route:
    - destination:
        host: reviews
        subset: v3
```

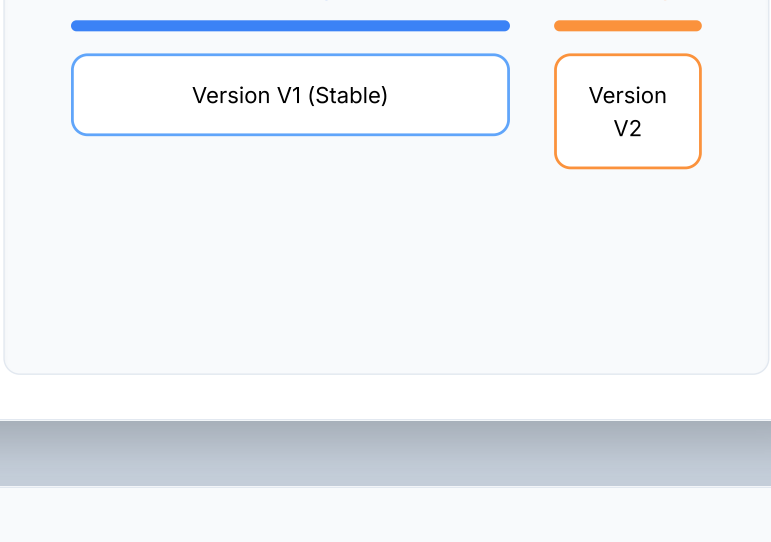
13. Déploiement Canary & A/B Testing

Division par pourcentage de "Poids"

Au lieu de router par Header ou URL, le Virtual Service peut répartir le trafic mathématiquement.

✓ Idéal pour tester une nouvelle version de votre application en production avec un rayon d'impact minimal (ex: 25% du trafic).

Scalabilité indépendante : Kubernetes gère le nombre de pods selon la charge, Istio gère qui y accède.



14. Code en action : Routage Canary

```yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews-canary
spec:
  hosts:
  - reviews
  http:
  # Pas de bloc "match" : cette règle s'applique à TOUT le trafic entrant
  - route:
    - destination:
        host: reviews
        subset: v1
      # Envoy Proxy enverra exactement 75% du trafic vers la V1
      weight: 75

    - destination:
        host: reviews
        subset: v2
      # Et les 25% restants iront tester la nouvelle release V2
      weight: 25
```

15. Destination Rules : La Gestion à l'Arrivée

> Appliquées **après** le Virtual Service.

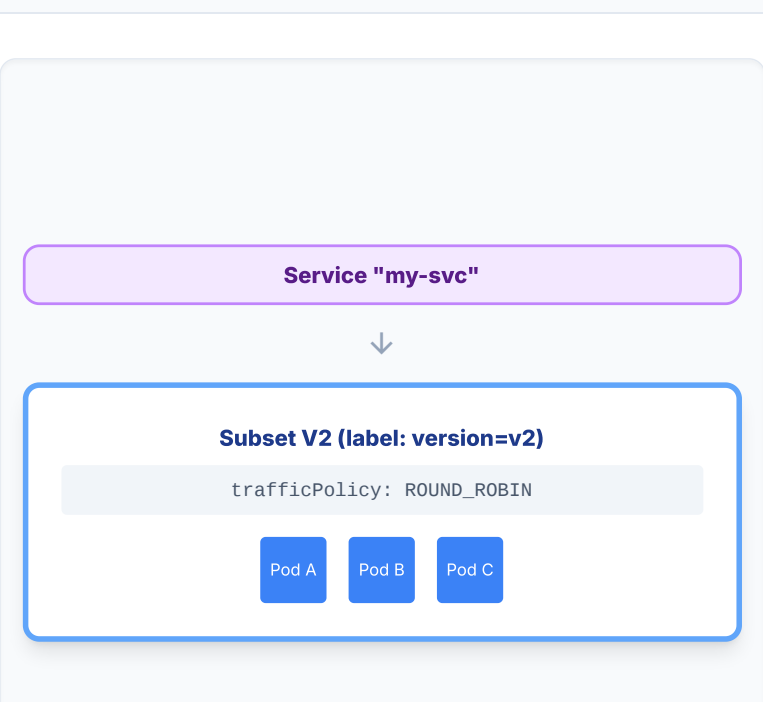
1. Création des Subsets :

La règle lit les metadata (labels) de vos pods Kubernetes pour créer les fameux sous-groupes (v1, v2) appelés par le Virtual Service.

2. Politique de Load Balancing :

Définit comment répartir la charge entre les pods du subset :

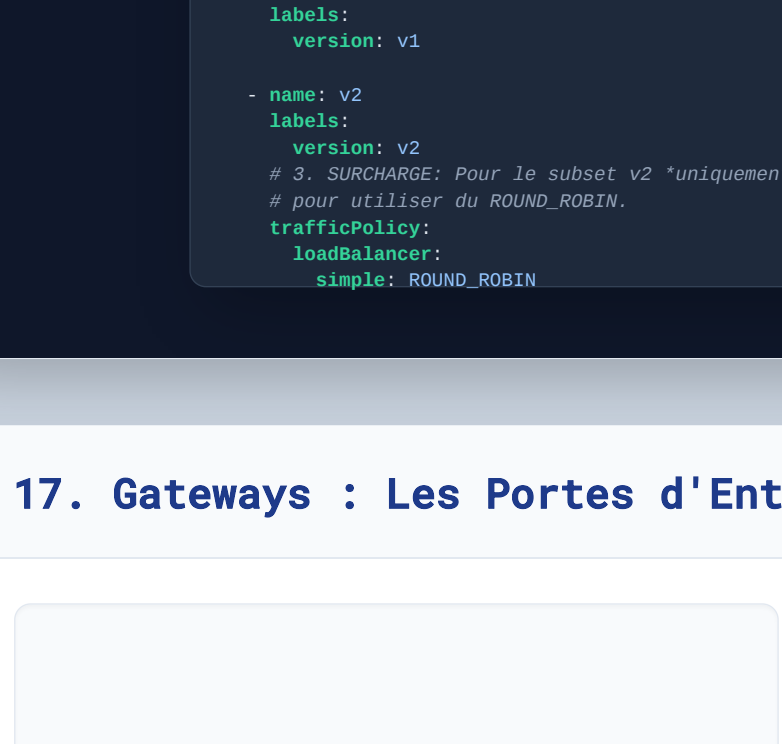
- **Least Requests (Défaut) :** Envoie vers le pod le moins occupé.
- **Random / Round Robin :** Distribution classique.
- **Consistent Hash :** Garantit la "Sticky Session".



16. Code en action : Destination Rule

```yaml
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: my-destination-rule
spec:
  host: my-svc

  # 1. RÈGLE GLOBALE: Par défaut, distribue les requêtes au hasard (RANDOM)
  trafficPolicy:
    loadBalancer:
      simple: RANDOM

  subsets:
  # 2. SUBSETS: Crée le groupe "v1" en lisant les labels Kubernetes des pods
  - name: v1
    labels:
      version: v1

  - name: v2
    labels:
      version: v2
    # 3. SURCHARGE: Pour le subset v2 *uniquement*, on écrase la règle globale
    # pour utiliser du ROUND_ROBIN.
    trafficPolicy:
      loadBalancer:
        simple: ROUND_ROBIN
```

17. Gateways : Les Portes d'Entrée (Ingress)



Gérer les frontières (L4-L6)

- Gèrent le trafic entrant (Ingress) et sortant (Egress).
- Ce sont des proxys Envoy **autonomes** exécutés à la frontière du cluster.
- La ressource Gateway configure uniquement la partie physique : le port (ex: 443) et le certificat TLS/SSL.
- Elle doit **obligatoirement** être liée à un Virtual Service pour comprendre l'URL demandée.

18. Code en action : Gateway & Binding

1. DÉCLARATION DE LA GATEWAY

```yaml
apiVersion: networking.istio.io/v1
kind: Gateway
metadata:
  name: ext-host-gwy
spec:
  selector:
    # Cible le Pod Envoy de bordure
    app: my-gateway-controller
  servers:
  - port:
      number: 443
      name: https
      protocol: HTTPS
    hosts:
    - ext-host.example.com
    tls:
      mode: SIMPLE
      # Certificat pour décrypter le flux TLS
      credentialName: ext-host-cert
```

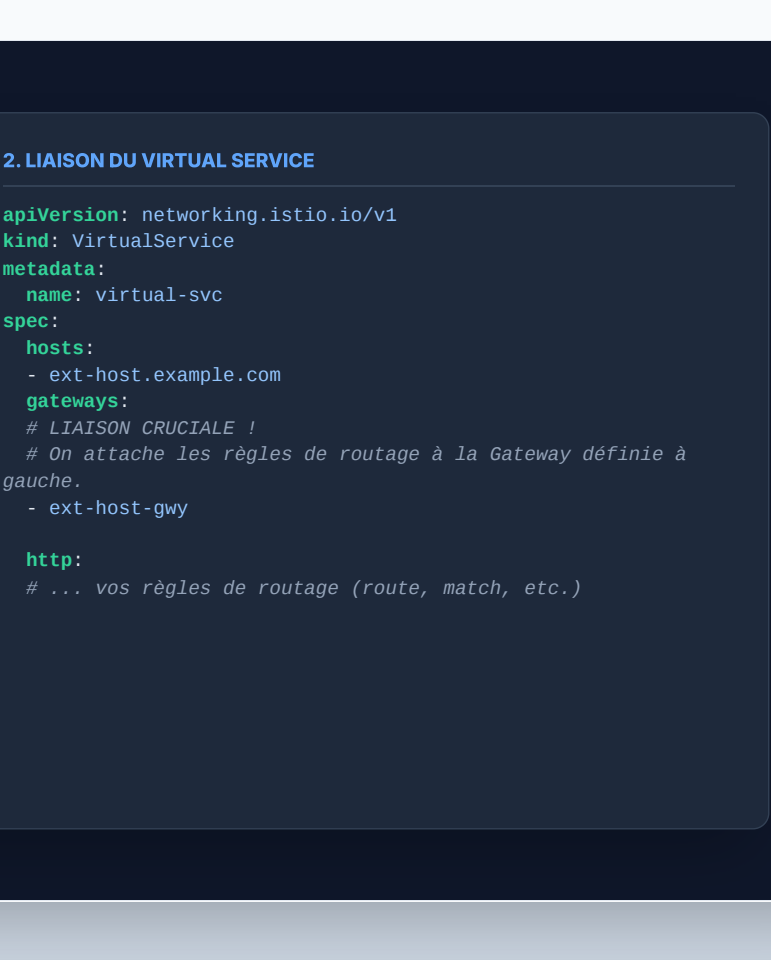
2. LIAISON DU VIRTUAL SERVICE

```yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: virtual-svc
spec:
  hosts:
  - ext-host.example.com
  gateways:
  # LIAISON CRUCIALE !
  # On attache les règles de routage à la Gateway définie à
gauche.
  - ext-host-gwy

  http:
  # ... vos règles de routage (route, match, etc.)
```

19. Service Entries : Gérer l'Externe

Par défaut, Istio ne connaît que les services à l'intérieur de votre cluster Kubernetes.

- Le Service Entry ajoute manuellement une API externe (ex: Base de données legacy) dans le registre interne d'Istio.
- Cela permet de traiter ce service externe comme un composant local et d'y appliquer vos règles de timeouts et retries.



20. Code en action : Service Entry

1. DÉCLARATION DU SERVICE EXTERNE

```yaml
apiVersion: networking.istio.io/v1
kind: ServiceEntry
metadata:
  name: svc-entry
spec:
  hosts:
  - ext-svc.example.com
  ports:
  - number: 443
    name: https
    protocol: HTTPS
  # Indique explicitement que la ressource vit hors du cluster
  location: MESH_EXTERNAL
  resolution: DNS
```

2. UTILISATION (APPLICATION DE POLITIQUES)

```yaml
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: ext-res-dr
spec:
  # On cible le nom défini dans le ServiceEntry
  host: ext-svc.example.com
  trafficPolicy:
    connectionPool:
      tcp:
        # On impose un timeout strict (1s) pour ne pas
        # geler notre application si l'API externe tombe.
        connectTimeout: 1s
```

21. Résilience : Timeouts & Retries

Timeouts

Coupe la connexion si le backend met trop de temps à répondre. Empêche les services de geler en cascade en attendant une réponse qui ne viendra jamais.

`Configuré dans le Virtual Service`

Retries

Réessaie automatiquement une connexion échouée pour masquer les micro-coupures réseau (ex: surcharge temporaire) sans que l'utilisateur ne s'en rende compte.

`Intervalle automatique intelligent (25ms+)`

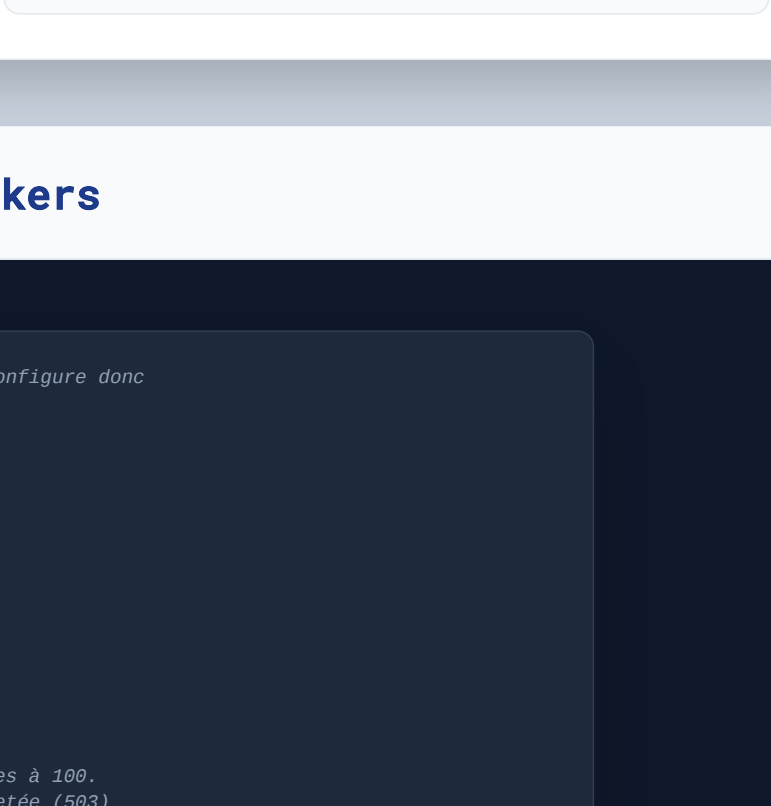
22. Code en action : Timeouts & Retries

```yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
  - ratings
  http:
  - route:
    - destination:
        host: ratings
        subset: v1

    # TIMEOUTS : Si "ratings" ne répond pas dans les 10s, la connexion échoue.
    timeout: 10s

    # RETRIES : Paramètres des tentatives automatiques.
    retries:
      # Envoy essaiera 3 fois au maximum de contacter le service.
      attempts: 3
      # Chaque tentative individuelle aura un délai strict de 2 secondes.
      perTryTimeout: 2s
```

23. Circuit Breakers : Isoler les pannes

Protéger le backend

- Définit des limites strictes (ex: *maxConnections: 100*).
- Si la limite est atteinte, le proxy "disjoncte" immédiatement.
- Évite de saturer un pod déjà en difficulté et force un échec rapide (Fail Fast) avec une Erreur 503.



24. Code en action : Circuit Breakers

```yaml
# Le Circuit Breaker protège la destination, il se configure donc
# logiquement dans la Destination Rule.
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: reviews
spec:
  host: reviews
  subsets:
  - name: v1
    labels:
      version: v1
    trafficPolicy:
      connectionPool:
        tcp:
          # Limite le nombre de connexions simultanées à 100.
          # La 101ème requête sera immédiatement rejetée (503)
          # pour protéger les pods v1 de la surcharge.
          maxConnections: 100
```

25. Fault Injection (Chaos Engineering)

Simuler les pires conditions

Plutôt que de débrancher un câble physique, Istio injecte les erreurs au niveau de la couche applicative pour observer les réactions de vos politiques de résilience.

Tester la robustesse en production

- Injecter des erreurs volontairement (et sous contrôle) au niveau applicatif.
- **Delays :** Simuler une latence réseau pour valider que vos *Timeouts* s'activent correctement.
- **Aborts :** Forcer des crashs (ex: renvoyer HTTP 500) pour tester si vos *Retries* et *Circuit Breakers* protègent le système.

26. Code en action : Fault Injection

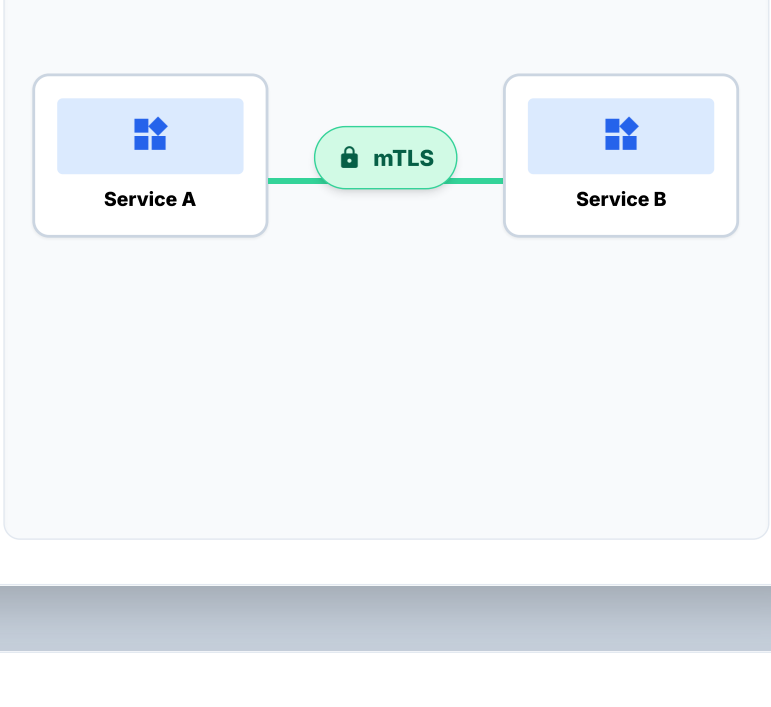
```yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: ratings
spec:
  hosts:
  - ratings
  http:
  - fault:
      # INJECTION D'UN DÉLAI ARTIFICIEL
      delay:
        percentage:
          # On impacte de manière aléatoire 0.1% des requêtes
          value: 0.1
        # Les requêtes ciblées subiront un délai forcé de 5 secondes
        fixedDelay: 5s
    route:
    - destination:
        host: ratings
        subset: v1
```

27. Sécurité : Zero-Trust avec le mTLS

Chiffrement Mutuel

Chiffrement automatique de tout le trafic entre les microservices via les proxys Envoy.

- Istio gère automatiquement la création, la rotation et l'échange des certificats X.509.
- Aucune gestion de clés requise dans le code métier de vos applications.



Questions & Réponses ?